

LIBRERIAS

```
!pip install pennylane
# Importación de librerías necesarias para la computación cuántica,
# análisis de datos y métricas de evaluación
import pennylane as qml
from pennylane import numpy as np
from pennylane.optimize import AdamOptimizer

from sklearn.model_selection import train_test_split
import pandas as pd

from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score
import math
```

```
Requirement already satisfied: pennylane in
/usr/local/lib/python3.11/dist-packages (0.41.1)
Requirement already satisfied: numpy in
/usr/local/lib/python3.11/dist-packages (from pennylane) (2.0.2)
Requirement already satisfied: scipy in
/usr/local/lib/python3.11/dist-packages (from pennylane) (1.15.3)
Requirement already satisfied: networkx in
/usr/local/lib/python3.11/dist-packages (from pennylane) (3.5)
Requirement already satisfied: rustworkx<=0.14.0 in
/usr/local/lib/python3.11/dist-packages (from pennylane) (0.16.0)
Requirement already satisfied: autograd in
/usr/local/lib/python3.11/dist-packages (from pennylane) (1.8.0)
Requirement already satisfied: tomlkit in
/usr/local/lib/python3.11/dist-packages (from pennylane) (0.13.3)
Requirement already satisfied: appdirs in
/usr/local/lib/python3.11/dist-packages (from pennylane) (1.4.4)
Requirement already satisfied: autoray<=0.6.11 in
/usr/local/lib/python3.11/dist-packages (from pennylane) (0.7.2)
Requirement already satisfied: cachetools in
/usr/local/lib/python3.11/dist-packages (from pennylane) (5.5.2)
Requirement already satisfied: pennylane-lightning<=0.41 in
/usr/local/lib/python3.11/dist-packages (from pennylane) (0.41.1)
Requirement already satisfied: requests in
/usr/local/lib/python3.11/dist-packages (from pennylane) (2.32.3)
Requirement already satisfied: typing-extensions in
/usr/local/lib/python3.11/dist-packages (from pennylane) (4.14.1)
Requirement already satisfied: packaging in
/usr/local/lib/python3.11/dist-packages (from pennylane) (24.2)
Requirement already satisfied: diastatic-malt in
/usr/local/lib/python3.11/dist-packages (from pennylane) (2.15.2)
Requirement already satisfied: scipy-openblas32<=0.3.26 in
/usr/local/lib/python3.11/dist-packages (from pennylane-
```

```
lightning>=0.41->pennylane) (0.3.30.0.1)
Requirement already satisfied: astunparse in
/usr/local/lib/python3.11/dist-packages (from diastatic-malt-
>pennylane) (1.6.3)
Requirement already satisfied: gast in /usr/local/lib/python3.11/dist-
packages (from diastatic-malt->pennylane) (0.6.0)
Requirement already satisfied: termcolor in
/usr/local/lib/python3.11/dist-packages (from diastatic-malt-
>pennylane) (3.1.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests->pennylane)
(3.4.2)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/dist-packages (from requests->pennylane)
(3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests->pennylane)
(2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->pennylane)
(2025.7.9)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.11/dist-packages (from astunparse->diastatic-
malt->pennylane) (0.45.1)
Requirement already satisfied: six<2.0,>=1.6.1 in
/usr/local/lib/python3.11/dist-packages (from astunparse->diastatic-
malt->pennylane) (1.17.0)
```

I. PREPROCESAMIENTO DE LOS DATOS DE ENTRADA

```
import zipfile
import os

with zipfile.ZipFile('titanic.zip', 'r') as zip_ref: # Descomprimir el
dataset Titanic
    zip_ref.extractall('titanic_data')

print("Archivos extraídos:") # Mostrar archivos extraídos
print(os.listdir('titanic_data'))

# Cargar y preparar los datos
# Convertimos las variables categóricas a variables "dummy"
# Rellenamos valores faltantes en la edad
# Creamos una nueva característica: si es niño o no. Con base en "age"

df_train = pd.read_csv('/content/titanic_data/train.csv')
```

```

df_train['Pclass'] = df_train['Pclass'].astype(str)
df_train = pd.concat([df_train, pd.get_dummies(df_train[['Pclass',
'Sex', 'Embarked']]]), axis=1)
df_train['Age'] = df_train['Age'].fillna(df_train['Age'].median())
df_train['is_child'] = df_train['Age'].map(lambda x: 1 if x < 12 else
0)
# Definimos las características (features) que vamos a usar
cols_model = ['is_child', 'Pclass_1', 'Pclass_2', 'Sex_female']

Archivos extraídos:
['train.csv', 'gender_submission.csv', 'test.csv']

```

II. DIVISIÓN DE LOS DATOS DE ENTRENAMIENTO Y DATOS DE PRUEBA

```

# Separamos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    df_train[cols_model],
    df_train['Survived'],
    test_size=0.10,
    random_state=42,
    stratify=df_train['Survived']
)
# Convertimos a formato compatible con PennyLane (tensores de
autograd)
X_train = np.array(X_train.values, requires_grad=False)
Y_train = np.array(y_train.values * 2 - np.ones(len(y_train)),
requires_grad=False)

```

III. ETAPA DEL FEATURE MAP

```

# Para la codificación de entrada clásica a cuántica (Feature Map)
def statepreparation(x):
    qml.BasisEmbedding(x, wires=range(0, 4))

```

IV. ETAPA DEL ANSATZ

```

# Definimos una capa del ansatz (bloques rotacionales +
entrelazamiento con CNOTs)
def layer(W):
    for i in range(4):
        qml.Rot(*W[i], wires=i)
    qml.CNOT(wires=[0, 1])

```

```

qml.CNOT(wires=[1, 2])
qml.CNOT(wires=[2, 3])
qml.CNOT(wires=[3, 0])

# Definimos el número de qubits y capas del circuito
num_qubits = 4
num_layers = 2
# Inicializamos el dispositivo cuántico
dev = qml.device("default.qubit", wires=num_qubits)

# Para la construcción del circuito cuántico completo con Feature Map
+ Ansatz
@qml.qnode(dev, interface="autograd")
def circuit(weights, x):
    statepreparation(x)
    for W in weights:
        layer(W)
    return qml.expval(qml.PauliZ(0)) # Medimos vaor esperado en Z del
primer qubit

# Clasificador cuántico variacional: suma del circuito y un bias
def variational_classifier(weights, bias, x):
    return circuit(weights, x) + bias

```

V. ETAPA DE FUNCIÓN DE PERDIDA

```

# Función de pérdida cuadrática
# Mide la diferencia entre predicción y etiqueta real

def square_loss(labels, predictions):
    loss = 0
    for l, p in zip(labels, predictions):
        loss += (l - p) ** 2
    return loss / len(labels)

```

VI. ETAPA DE EVALUACIÓN DE LA FUNCIÓN DE COSTO

```

# Función de costo que depende de los parámetros y los datos
# Utiliza square_loss para calcular el error

def cost(weights, bias, X, Y):
    predictions = [variational_classifier(weights, bias, x) for x in
X]
    return square_loss(Y, predictions)

```

VII. ETAPA DE ANÁLISIS DE PRECISIÓN

```
# Cálculo de accuracy (exactitud)

def accuracy(labels, predictions):
    correct = 0
    for l, p in zip(labels, predictions):
        if abs(l - p) < 1e-5:
            correct += 1
    return correct / len(labels)
```

VIII. PESOS INICIALES DEFINIDOS

```
# Inicializamos los pesos del circuito y el bias. La de los pesos se hace de manera pseudoaleatoria

np.random.seed(0)
weights_init = 0.01 * np.random.randn(num_layers, num_qubits, 3, requires_grad=True)
bias_init = np.array(0.0, requires_grad=True)

weights = weights_init
bias = bias_init
print(weights_init)
print(bias_init)

[[[ 0.01764052  0.00400157  0.00978738]
  [ 0.02240893  0.01867558 -0.00977278]
  [ 0.00950088 -0.00151357 -0.00103219]
  [ 0.00410599  0.00144044  0.01454274]]

  [[ 0.00761038  0.00121675  0.00443863]
  [ 0.00333674  0.01494079 -0.00205158]
  [ 0.00313068 -0.00854096 -0.0255299 ]
  [ 0.00653619  0.00864436 -0.00742165]]]
0.0
```

IX. PESOS FINALES ENTRENADOS

```
opt = AdamOptimizer(0.125) # Optimizador clásico (AdamOptimizer)
num_it = 70
batch_size = math.floor(len(X_train) / num_it)

# Entrenamiento del modelo usando batches aleatorios
for it in range(num_it):
    batch_index = np.random.randint(0, len(X_train), (batch_size,))
```

```

#indexación para elegir batch aleatorio
X_batch = X_train[batch_index]
Y_batch = Y_train[batch_index]
weights, bias, _, _ = opt.step(cost, weights, bias, X_batch,
Y_batch) # Optimización por paso

    predictions = [np.sign(variational_classifier(weights, bias, x))
for x in X_train]
    acc = accuracy(Y_train, predictions) # Medición de accuracy en el
set de entrenamiento

    print(f"Iter: {it + 1:5d} | Cost: {cost(weights, bias, X_train,
Y_train):.7f} | Accuracy: {acc:.7f}")

# parámetros finales aprendidos
print(weights)
print(bias)

```

Iter:	1	Cost: 2.3009054	Accuracy: 0.3657928
Iter:	2	Cost: 2.0157333	Accuracy: 0.3657928
Iter:	3	Cost: 1.6928554	Accuracy: 0.3657928
Iter:	4	Cost: 1.4394783	Accuracy: 0.3657928
Iter:	5	Cost: 1.2992964	Accuracy: 0.5205993
Iter:	6	Cost: 1.2592430	Accuracy: 0.6167291
Iter:	7	Cost: 1.2860673	Accuracy: 0.6167291
Iter:	8	Cost: 1.3340117	Accuracy: 0.6167291
Iter:	9	Cost: 1.3157367	Accuracy: 0.6167291
Iter:	10	Cost: 1.2278186	Accuracy: 0.6167291
Iter:	11	Cost: 1.1070846	Accuracy: 0.6167291
Iter:	12	Cost: 1.0319251	Accuracy: 0.6167291
Iter:	13	Cost: 0.9932298	Accuracy: 0.6167291
Iter:	14	Cost: 0.9745684	Accuracy: 0.6167291
Iter:	15	Cost: 0.9807690	Accuracy: 0.7802747
Iter:	16	Cost: 0.9487227	Accuracy: 0.7802747
Iter:	17	Cost: 0.9169272	Accuracy: 0.7802747
Iter:	18	Cost: 0.8685417	Accuracy: 0.7802747
Iter:	19	Cost: 0.8121101	Accuracy: 0.7802747
Iter:	20	Cost: 0.7734251	Accuracy: 0.7802747
Iter:	21	Cost: 0.7578252	Accuracy: 0.7802747
Iter:	22	Cost: 0.7533380	Accuracy: 0.7802747
Iter:	23	Cost: 0.7540889	Accuracy: 0.7727840
Iter:	24	Cost: 0.7547410	Accuracy: 0.7727840
Iter:	25	Cost: 0.7418572	Accuracy: 0.7727840
Iter:	26	Cost: 0.7216730	Accuracy: 0.7727840
Iter:	27	Cost: 0.7054738	Accuracy: 0.7840200
Iter:	28	Cost: 0.6913697	Accuracy: 0.7840200
Iter:	29	Cost: 0.6826785	Accuracy: 0.7840200
Iter:	30	Cost: 0.6772999	Accuracy: 0.7840200
Iter:	31	Cost: 0.6758436	Accuracy: 0.7840200
Iter:	32	Cost: 0.6825518	Accuracy: 0.7840200

Iter:	33	Cost:	0.6856851	Accuracy:	0.7840200
Iter:	34	Cost:	0.6896622	Accuracy:	0.7840200
Iter:	35	Cost:	0.6943790	Accuracy:	0.7840200
Iter:	36	Cost:	0.7012850	Accuracy:	0.7840200
Iter:	37	Cost:	0.7067733	Accuracy:	0.7840200
Iter:	38	Cost:	0.7103192	Accuracy:	0.7840200
Iter:	39	Cost:	0.7161690	Accuracy:	0.7840200
Iter:	40	Cost:	0.7176914	Accuracy:	0.7840200
Iter:	41	Cost:	0.7134635	Accuracy:	0.7840200
Iter:	42	Cost:	0.7095454	Accuracy:	0.7840200
Iter:	43	Cost:	0.7075034	Accuracy:	0.7840200
Iter:	44	Cost:	0.7016297	Accuracy:	0.7840200
Iter:	45	Cost:	0.6959733	Accuracy:	0.7840200
Iter:	46	Cost:	0.6963289	Accuracy:	0.7840200
Iter:	47	Cost:	0.6943661	Accuracy:	0.7840200
Iter:	48	Cost:	0.6855695	Accuracy:	0.7840200
Iter:	49	Cost:	0.6820337	Accuracy:	0.7840200
Iter:	50	Cost:	0.6739566	Accuracy:	0.7840200
Iter:	51	Cost:	0.6727428	Accuracy:	0.7840200
Iter:	52	Cost:	0.6757914	Accuracy:	0.7840200
Iter:	53	Cost:	0.6806343	Accuracy:	0.7840200
Iter:	54	Cost:	0.6841203	Accuracy:	0.7840200
Iter:	55	Cost:	0.6862026	Accuracy:	0.7840200
Iter:	56	Cost:	0.6912584	Accuracy:	0.7840200
Iter:	57	Cost:	0.6992686	Accuracy:	0.7852684
Iter:	58	Cost:	0.7072007	Accuracy:	0.7852684
Iter:	59	Cost:	0.7130674	Accuracy:	0.7852684
Iter:	60	Cost:	0.7230222	Accuracy:	0.7852684
Iter:	61	Cost:	0.7195306	Accuracy:	0.7852684
Iter:	62	Cost:	0.7081696	Accuracy:	0.7852684
Iter:	63	Cost:	0.7017707	Accuracy:	0.7852684
Iter:	64	Cost:	0.6935658	Accuracy:	0.7840200
Iter:	65	Cost:	0.6855921	Accuracy:	0.7840200
Iter:	66	Cost:	0.6818968	Accuracy:	0.7840200
Iter:	67	Cost:	0.6849336	Accuracy:	0.7840200
Iter:	68	Cost:	0.6827583	Accuracy:	0.7840200
Iter:	69	Cost:	0.6800306	Accuracy:	0.7840200
Iter:	70	Cost:	0.6809625	Accuracy:	0.7840200

```

[[[ 1.76405273e-02  1.62072071e+00  4.53474515e-01]
   [ 2.24089342e-02  1.94456302e+00 -2.15431674e+00]
   [ 9.50088407e-03  1.20709542e+00  1.16574675e-01]
   [ 4.10598622e-03  4.34849692e-01 -5.29636512e-02]]

[[ 7.61037848e-03  1.21675062e-03  4.43863356e-03]
 [-1.04693033e-01  1.44260145e+00 -2.05157958e-03]
 [ 1.49158955e+00 -2.13282131e+00 -2.55298970e-02]
 [ 1.84665689e+00  1.95568097e+00 -7.42165150e-03]]]
-0.19042059623599075

```

X. EVALUACIÓN DE DESEMPEÑO DEL CLASIFICADOR CUÁNTICO FINAL

```
X_test = np.array(X_test.values, requires_grad=False)
Y_test = np.array(y_test.values * 2 - np.ones(len(y_test)),
requires_grad=False)

predictions = [np.sign(variational_classifier(weights, bias, x)) for x
in X_test]

# Métricas de evaluación
print("Accuracy:", accuracy_score(Y_test, predictions))
print("Precision:", precision_score(Y_test, predictions))
print("Recall:", recall_score(Y_test, predictions))
print("F1 Score:", f1_score(Y_test, predictions, average='macro'))

Accuracy: 0.7888888888888889
Precision: 0.7666666666666667
Recall: 0.6571428571428571
F1 Score: 0.7712374581939799
```